

# 끝가지 않는 흐름을 설계하는 백엔드 개발자

기능을 “만드는” 데서 멈추지 않고, 사용자의 요청이 인증부터 실시간 상태, 데이터 저장, 배포까지 **한 번도 어긋나지 않고 이어지도록** 설계합니다. 특히 여러 사용자가 동시에 오가는 실시간 시스템에서 “상태가 깨지지 않는 흐름”을 만드는 데 집중해 왔습니다.

[github.com/TaeHeeLee0](#)   [k58701@naver.com](#)   [인천 · 수도권 · 근무 희망](#)

|                          |                       |                 |                        |
|--------------------------|-----------------------|-----------------|------------------------|
| <b>4</b><br>SSAFY 팀 프로젝트 | <b>1</b><br>최우수상 · 1등 | <b>2</b><br>우수상 | <b>1년</b><br>실무형 개발 경험 |
|--------------------------|-----------------------|-----------------|------------------------|

## 저는 이런 개발자입니다

화려한 기능보다, 오래 안정적으로 동작하고 문제를 추적할 수 있으며 팀이 계속 발전시킬 수 있는 시스템을 지향합니다.

|                                                                                                               |                                                                                                                        |
|---------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| <b>실제를 먼저 설계합니다</b><br>정상 경로보다 장애·예외 상황을 먼저 상상합니다.<br>서버가 재시작되거나 요청이 물려도 데이터 상태가 어긋나지 않도록 정합성과 동시성 제어를 우선합니다. | <b>필요한 기술을 빠르게 익힙니다</b><br>필요를 예측해 미리 준비하고, 처음 다루는 기술도 짧은 시간에 실전 적용합니다. 처음 쓰는 Jenkins로 자동 배포 환경을 하루 만에 구축한 경험이 대표적입니다. |
|---------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|

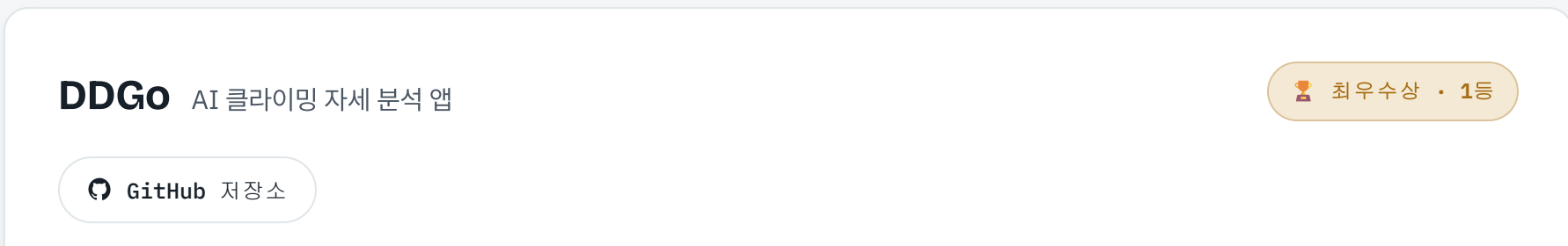
## 기술 스택

실제 프로젝트에서 문제를 해결하는 데 직접 사용한 기술입니다.

|                                                                                                                                         |                                                                                                                  |
|-----------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| <b>Language &amp; Framework</b><br>도메인-관련자 API와 실시간 서버를 구현한 주력 스택<br><b>Java</b> · <b>Spring Boot</b> · Spring Security · JPA · FastAPI | <b>Realtime &amp; Cache</b><br>실시간 상태 동기화와 세션-분산 락 처리에 사용<br><b>WebSocket</b> · <b>Redis</b> · STOMP · WebRTC    |
| <b>Database</b><br>사용자-기록-콘텐츠 데이터 설계와 조회 성능 개선<br><b>MySQL</b> · <b>PostgreSQL</b> · MariaDB · SQL                                      | <b>Infra &amp; DevOps</b><br>배포 자동화와 실행 환경 구성, 대용량 파일 저장소<br><b>Docker</b> · <b>Jenkins</b> · <b>AWS</b> · MinIO |

## 흐름을 끝까지 연결합니다

사용자의 요청 하나가 인증부터 실시간 처리, 저장, 배포까지 끝까지 않고 이어지도록, 서비스의 전 구간을 직접 다뤄 왔습니다.



## 프로젝트

정상 경로에서는 드러나지 않아 원인을 파고들어야 보이는 문제들을 골라, 어떻게 발견하고 해결했는지를 중심으로 정리했습니다. 모든 수치는 명시된 측정 조건 기준입니다.

**DDGo** AI 클라이언팅 자체 분석 앱

[GitHub 저장소](#)

기간 2026.02.16 ~ 04.03   인원 6명   역할 **백엔드 중심 풀스택**

담당 인증·운동기록 API·영상 업로드·AI 분석 연동·위치 심박 연동·배포 자동화

Java · Spring Boot · Redis · MariaDB · MinIO · Docker · Jenkins · FastAPI · Wear OS

클라이언팅 운동 영상과 스마트워치 심박 데이터를 분석해 자세를 평가·기록하는 모바일 서비스. 앱·위치·서버·영상 저장소(Minio presigned 업로드)·AI 분석·자동 배포까지 end-to-end로 구성했습니다.

**기술 선택** 대용량 영상은 MinIO 직접 업로드로 서버 부하를 분기하고, 인증·세션 상태는 Redis로 서버가 직접 통제하며, 반복되는 배포는 Jenkins로 자동화해 팀 개발 속도를 확보했습니다.

**DDGo**

**AI가 분석하는 클라이언팅 시도 기록**

영상 업로드와 실시간 녹화로 홀드 탐지, 자세 분석, 신체 부하 피드백까지

홀드 탐지 · 자세 분석 · 신체 부하

위치 심박 연동 · 홀드 AI 탐지 · 실시간 녹화 분석 · 신체 부하 부위별 분석 · 커뮤니티 영상 공유

AI가 분석하는 클라이언팅 시도 기록 (홀드 탐지 · 자세 분석 · 신체 부하)

**Client Layer**

Android App Jetpack Compose

On-Device AI

AI Models

MediaPipe Pose Detector

ultralytics YOLOv8 Person Detector

ultralytics YOLOv11 Hold Detector

CI/CD Pipeline

GitHub

Jenkins

Docker

**AWS EC2 Environment**

GATEWAY

NGINX Reverse Proxy

Docker Engine / Compose

Spring Boot Main Backend API

FastAPI AI Inference Server

Physics Engine Advanced Simulation

MariaDB RDBMS

Redis In-Memory Cache

MINIO Object Storage

시스템 아키텍처 · 온디바이스 AI · Spring Boot · FastAPI (MuJoCo) · MinIO/Redis/MariaDB · GitHub → Jenkins → Docker CI/CD

**문제 해결**

**배포 환경에서 터지는 저장소 주소 불일치 해결**

**문제** 로컬에서는 멀쩡하던 영상 조회·다운로드가 **배포 환경에서만 실패**했습니다. MinIO가 Docker 내부 네트워크에서 쓰는 주소와, 외부 클라이언트가 접근해야 하는 공개 주소가 서로 달랐던 것이 원인였고, 정상 개발 흐름에서는 드러나지 않는 문제였습니다.

**해결** 서버가 저장소에 접근하는 내부 경로와, 클라이언트에게 내려주는 외부 경로를 분리하고, 발급되는 URL을 외부 공개 주소로 재작성(rewriting)하는 로직을 넣었습니다.

**결과** Docker 내부망과 외부 접근 경로가 다른 실제 배포 환경에서도 영상 흐름이 끊기지 않도록 했습니다.

**처음 쓰는 Jenkins로 자동 배포를 하루 만에 구축**

**문제** 배포할 서버가 없어 로컬로만 개발했고, 서버 지급 후에도 수동 배포는 느린 데다 매번 전체를 다시 빌드했습니다.

**해결** 배포 환경 필요를 미리 예상해 학습해 둔 덕분에 처음 쓰는 Jenkins로 자동 배포를 **하루 만에** 구축. 나아가 Dockerfile 안에서 Gradle 빌드를 수행하던 단계를 제거하고 Jenkins가 만든 실행 JAR을 복사하도록 파이프라인을 재설계했습니다.

**결과** 코드가 올리면 자동 빌드·배포되고 실패는 팀 메신저로 즉시 공유. **clean 빌드 17.81s → 11.93s (약 33%↓)**  
측정: 로컬 재현 기준 (운영 서버 빌드 시간과 다를 수 있음)

**Play! Nemonic** 실시간 드로잉 콘텐츠 제작·공유 서비스

[GitHub 저장소](#)

기간 2026.04.28 ~ 05.27   인원 6명   역할 **백엔드 중심 풀스택**

담당 실시간 필레이 서버·이미지 파이프라인·동시성·정합성 제어·구조 개선

Java · Spring Boot · WebSocket · Redis · PostgreSQL · JPA · MinIO · FastAPI · JUnit / Testcontainers

사용자가 그린 그림을 커뮤니티 게시물·릴레이 드로잉·플린북으로 만들고, 프린터 출력·QR 공유로 잇는 참여형 서비스. 네 프로젝트 중 규모가 가장 크고 실시간 시스템 설계 역량을 크게 키운 경험입니다.

**기술 선택** 여러 사용자의 실시간 상태를 빠르게 공유하고 동시성을 제어하기 위해 Redis 기반 상태·분산 락을, 지연 없는 양방향 전송을 위해 WebSocket을 선택했습니다.

**Play! nemonic**

PLAY NEMONIC

다양한 콘텐츠로 기기 없이도 네모닉 직접 경험

무한캔버스, 플립북, 릴레이 게임, 오늘의 문제와 커뮤니티까지 즐기며 출력 소리와 시각각을 화면에서 느껴보세요

무한캔버스 · 플립북 · 릴레이 · 네모닉 경험 · 오늘의 문제 · 3D 프린팅

기기 없이도 네모닉을 직접 경험하는 참여형 콘텐츠 (무한캔버스 · 플립북 · 릴레이 · 문제)

**문제 해결**

**서버 재시작 후 유령 방이 쌓이는 정합성 붕괴 해결**

**문제** 서버를 재시작하면 방지된 필레이 방이 영원히 닫히지 않았습니다. 접속 상태는 Redis에, 실제 연결은 서버 메모리에 있어, 서버가 죽으면 Redis에 “접속 중”이 그대로 남았기 때문입니다.

**해결** 테스트로는 잡히지 않는 문제였기에, “프로세스가 죽으면 인메모리 상태와 외부 상태가 어긋난다”는 가정을 세우고, 두 상태를 주기적으로 맞추는 정합성 (reconciliation) 스케줄러를 방 지령보다 먼저 돌도록 설계했습니다.

**결과** 재시작 후에도 유령 방이 쌓이지 않도록 상태 불일치를 자동 복구. 실패 경로를 먼저 설계하는 접근을 체득했습니다.

**분산 락의 ‘남의 락 삭제’ 합정까지 막은 동시성 제어**

**문제** 여러 사용자가 동시에 다른 파트를 그리고 제출·자동제출·최종화가 겹치면서, 상태 전이와 결과 생성에서 경합이 발생할 수 있었습니다.

**해결** Redis 낙관적 갱신(WATCH/MULTI/EXEC)과 제출·최종화 락을 적용하고, 락 해제에는 UUID 토큰을 비교 후 지우는 원자적 처리(Lua script)를 써서 남의 락을 지우는 문제까지 막았습니다.

**결과** 사용자 제출과 자동 제출이 같은 정합성 경계를 통과, 최종화를 scheduler 의존에서 이벤트 직후 비동기 트러거로 바꿔 **대기 시간 최대 10초 단축**.

**방지 방 정리의 N+1 조회를 배치 쿼리로 제거**

**문제** 방지된 방을 정리할 때 각 방의 DB 참조 여부를 받마다 개별 조회해, 한 번의 정리에 방 수에 비례한 수백 번의 쿼리가 발생할 수 있었습니다.

**해결** 방마다 반복하던 조회를, 필요한 대상을 한 번에 확인하는 배치 쿼리로 바꿔 정리 로직이 방 개수에 비례해 느려지지 않도록 했습니다.

**결과** **정리 시 DB 참조 확인 최대 500회 → 1회 (약 99.8%↓)**  
측정: 스캔 한도 500 기준

정리 시 DB 조회 수 -99.8%  
이전 최대 500회  
개선 1회

**EchoForest** AI 감성 인식 실시간 협동 게임엔

[GitHub 저장소](#)

기간 2026.01.06 ~ 02.13   인원 6명   역할 **백엔드 중심 풀스택**

담당 실시간 게임 서버·상태 동기화·접속/세션 관리

Java · Spring Boot · WebSocket · Redis · MySQL · WebRTC

네 명이 화상·음성으로 소통하며 함께 스테이지를 클리어하는 협동 웹게임. 플레이어의 말이 공격/부정인지 AI가 인식해 게임 진행에 영향을 줍니다. 플레이어의 상태가 실시간으로 어긋나지 않도록 하는 서버를 담당했습니다.

**기술 선택** 멀티플레이 위치·이벤트의 일관성이 핵심이라, 클라이언트가 각자 계산하는 대신 서버가 최종 상태를 계산하는 서버 권위형 구조를 택했습니다.

**메아리의 숲 Echo Forest**

부정어가 쌓이면 지주 스택 공중돌 푸는 협동 PLAY

목소리로 지주 스택을 관리하며 함께 스테이지를 클리어하는 웹 협동 게임 모션 미션 완료 후 추억 사진을 메일로 받아요.

지주 스택 · 메아리 전수 · 모션 미션 · 모험 동기 · 추억 메일

실시간 협동 웹게임 ‘메아리의 숲’ (부정어는 지주, 공중돌은 해적)

**문제 해결**

**서버 권위형 tick loop로 멀티플레이 상태 일관성 확보**

**문제** 서버와 클라이언트가 각각 위치를 계산해, 엘리베이터·굴안·스테이지 전환이 사람마다 다르게 보였습니다.

**해결** 서버 권위형(server-authoritative) tick loop를 기준으로 최종 상태를 계산하고, 클라이언트는 서버 상태를 신뢰하도록 예측 보간을 조정했습니다.

**결과** 캐릭터·오브젝트·이벤트의 일관성을 확보해 멀티플레이 동기화 안정성을 높였습니다.

**Redis 세션과 실제 연결의 불일치(유령 세션) 정리**

**문제** 중복 로그인·자동 로그아웃을 처리하다 보면, Redis에는 로그인 세션이 남아 있지만 실제 WebSocket 연결은 이미 끊긴 ‘유령 세션’이 생겨 ready 상태가 어긋나거나 유령 플레이어가 남았습니다.

**해결** 로그인 시 Redis 세션뿐 아니라 서버가 들고 있는 활성 WebSocket 세션까지 함께 확인해, 실제 연결이 없으면 남은 Redis 세션을 정리하도록 했습니다.

**결과** 자동 로그아웃 이후의 상태 불일치와 유령 플레이어가 문제를 줄여 실시간 방 상태를 안정화했습니다.

**상태 메시지 프로토콜을 재설계로 전송량 65% 감소**

**문제** 플레이어 상태를 객체 JSON으로 초당 수십 번 전송하다 보니 메시지 용량이 커져 실시간 반응성이 떨어졌습니다.

**해결** 필드 이름을 반복해 담은 방식 대신 값만 순서대로 담은 배열 프로토콜로 바꾸고, 서버 전송과 클라이언트 파싱을 모두 인메모리 기반으로 전환했습니다.

**결과** **상태 메시지 1,028B → 361B (약 65%↓)**. 전송 빈도를 초당 20 → 30회로 높인 뒤에도 총 전송량은 오히려 감소.  
측정: 4인 플레이 상태 메시지 본문 기준

상태 메시지 용량 -65%  
이전 1,028 B  
개선 361 B

## 그 외 활동

|                                                                                                                              |                                                                                            |
|------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <b>PetCare Box · 우수상(2등)</b><br>반려동물 통합 케어 플랫폼(SSAFY 팀 프로젝트). 데이터베이스 설계와 지도 기반 병원 예약, 오디오비시와 서버 AI를 결합한 하이브리드 상담 기능을 담당했습니다. | <b>알고리즘 · 서버 스타디</b><br>알고리즘 문제 풀이와 서버 개발 스타디를 지속하며 Java·Spring Boot 구현력과 문제 해결 속도를 높였습니다. |
| <b>협업 방식</b><br>이슈 기반 스프린트, 코드 리뷰(MR)와 브랜치 전략, 커밋 컨벤션을 지키며 팀이 믿고 함께 개발할 수 있는 코드를 남겼습니다.                                      | <b>학력</b><br>한신대학교 컴퓨터공학부 졸업 (학점 3.9 / 4.5) · SSAFY 1년 과정 수료.                              |

## 함께 일하고 싶은, 신뢰받는 백엔드 개발자

많은 서비스의 흐름을 끝까지 책임지겠습니다.

[github.com/TaeHeeLee0](#)